ASDD PRACTICALS
REVISION GUIDE

Experiments 1–8 • what to do, what to verify, what they ask

The whole subject in one sentence: plan work → version the code → build it → test it → ship it → run it in containers → monitor it → run code without servers.

Stage	Experiment	Tool
Plan	1 – Agile lifecycle	Jira
Code / version	2 – Git collaboration	Git + GitHub
Build / automate	3 – Jenkins install	Jenkins
CI/CD	4 – Jenkins + GitHub	Jenkins + GitHub
Orchestrate	5 – Deploy & scale	Kubernetes
Package	6 – Containers	Docker
Monitor	7 – Dashboards	Grafana
Serverless	8 – Function	AWS Lambda

Experiment 1 — Agile lifecycle in Jira

Aim: study and implement the Agile software project lifecycle using Jira with DevOps pipeline integration.

What you actually do

1. Create a free account at atlassian.com and open Jira.
2. Create a project → choose the Scrum template → name it.
3. Create an Epic, then create 3–4 Stories inside it.
4. Open the Backlog, drag stories into a Sprint, click Start Sprint.
5. Open the Board. Drag each story across To Do → In Progress → Done.
6. Show the List view and the Timeline view.
7. Invite a collaborator (Project settings → People → invite by email).
8. Complete the sprint and show the sprint report.

What to verify / screenshot

- Backlog with stories prioritised
- Active sprint board with cards in different columns
- Timeline view
- Sprint report

Viva: Q: What is a sprint? — a fixed time box (1–4 weeks) where a set of stories is completed Q: Story vs Epic? — story = small deliverable; epic = large body of work split into stories Q: Scrum vs Kanban? — Scrum = fixed sprints and roles; Kanban = continuous flow with WIP limits Q: What is the backlog? — the prioritised list of all work not yet in a sprint

Experiment 2 — Git version control

Aim: perform version control operations using Git and apply collaboration workflows.

Commands to run

```
git --version
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --list
mkdir GitProject && cd GitProject
git init
git status
echo "# GitProject" > README.md
git add README.md
git commit -m "Initial Commit"
git branch -M main
git remote add origin https://github.com/<user>/<repo>.git
git push -u origin main
```

Collaboration flow

```
git checkout -b feature-branch
# edit files
git add .
git commit -m "Add feature"
git push origin feature-branch
# open Pull Request on GitHub -> review -> merge
git checkout main
git pull origin main
```

What to verify / screenshot

- git status showing clean tree
- commit log (git log --oneline)
- GitHub repo with README and 2 branches
- merged pull request

Viva: Q: Git vs GitHub? — Git is the local version-control tool; GitHub is the remote hosting/collaboration service Q: commit vs push? — commit saves a snapshot locally; push uploads commits to the remote Q: What is a branch? — an independent line of development that can later be merged Q: pull vs fetch? — fetch downloads changes; pull fetches and merges them

Experiment 3 — Install Jenkins

Aim: install Jenkins, launch it on an available port and trigger a sample Hello World build.

Do this

9. Install Java (JDK 17 or 21) and verify with java -version.
10. Download jenkins.war from jenkins.io (or use the MSI/apt package).
11. Start it: java -jar jenkins.war --httpPort=8080 (choose a free port if 8080 is busy).
12. Open http://localhost:8080. Unlock with the initial admin password (printed in the terminal or found in ~/.jenkins/secrets/initialAdminPassword).
13. Install suggested plugins, create the admin user, finish setup.
14. New Item → Freestyle project → name it HelloWorld.
15. Build Steps → Execute shell (or Windows batch) → echo Hello World.
16. Save → Build Now → open Console Output.

What to verify / screenshot

- Jenkins dashboard
- Build #1 success
- Console Output showing Hello World

Viva: Q: What is Jenkins? — an open-source automation server used to build, test and deploy software Q: Why Java? — Jenkins itself runs on the JVM Q: What is a build job? — a configured task that Jenkins executes, e.g. compile, test or run a script Q: Where is the initial password? — ~/.jenkins/secrets/initialAdminPassword or the terminal output

Experiment 4 — Jenkins + GitHub CI/CD

Aim: integrate Jenkins with GitHub and simulate a CI/CD pipeline for automated building and testing.

Do this

17. Create a GitHub repository and push a small project with a Jenkinsfile.
18. GitHub → Settings → Developer settings → Personal access tokens → generate a token with repo scope.
19. Jenkins → Manage Jenkins → Credentials → add Username with password (username = GitHub user, password = token).
20. Manage Jenkins → Plugins → ensure Git and GitHub plugins are installed.
21. New Item → Pipeline → Pipeline script from SCM → Git → repo URL → credential → branch */main → Jenkinsfile.
22. Add a trigger: GitHub hook trigger, or Poll SCM with H/5 * * * *.
23. Optional webhook: GitHub repo → Settings → Webhooks → http://<jenkins>/github-webhook/ (use ngrok if Jenkins is local).
24. Build Now and watch the stages in Console Output / Blue Ocean.

Sample Jenkinsfile

```
pipeline {
 agent any
 stages {
  stage('Checkout') { steps { git 'https://github.com/<user>/<repo>.git' } }
  stage('Build')    { steps { echo 'Building...' } }
  stage('Test')     { steps { echo 'Running tests...' } }
  stage('Deploy')   { steps { echo 'Simulated deployment...' } }
 }
}
```

What to verify / screenshot

- Pipeline job configuration
- Stage view with all stages green
- A push on GitHub triggering a build (or Poll SCM log)

Viva: Q: CI vs CD? — CI = automatically build and test every change; CD = automatically deliver/deploy it Q: What is a Jenkinsfile? — a text file that defines the pipeline as code and is stored in the repo Q: Webhook vs polling? — webhook = GitHub notifies Jenkins immediately; polling = Jenkins checks on a schedule Q: Why a personal access token? — so Jenkins can authenticate to GitHub without using the account password

Experiment 5 — Kubernetes: deploy, expose, scale

Aim: deploy, expose and scale an application on a Kubernetes cluster.

Fastest option — no installation

25. Open https://killercoda.com and log in with GitHub.
26. Playgrounds → Kubernetes (1-node) → Start. The terminal opens with the cluster ready.
27. Run the commands below. Skip minikube start.

Commands

```
kubectl get nodes
kubectl create deployment nginx --image=nginx
kubectl get deployments
kubectl get pods
kubectl expose deployment nginx --type=NodePort --port=80
kubectl get svc
# note the port: 80:31234
curl localhost:<NodePort>
kubectl scale deployment nginx --replicas=3
kubectl get pods
kubectl describe deployment nginx
kubectl delete service nginx
kubectl delete deployment nginx
```

What to verify / screenshot

- node Ready
- deployment created
- service with NodePort
- curl prints Welcome to nginx!
- 3 pods Running after scale

Viva: Q: Pod vs Deployment? — a pod is the smallest runnable unit; a deployment manages pods and their replicas/updates Q: What is a Service? — a stable network endpoint that exposes pods Q: What is NodePort? — a service type that opens a port on every node so the app is reachable externally Q: How do you scale? — kubectl scale deployment <name> --replicas=N

Experiment 6 — Docker containers

Aim: develop and run an application using Docker containers.

Commands

```
git clone https://github.com/docker/getting-started-todo-app
cd getting-started-todo-app
docker compose watch
docker build -t welcome .
docker run -d -p 3000:3000 welcome
docker ps
```

What to verify / screenshot

- Docker Desktop showing the running container
- docker ps output
- app open in the browser
- image listed in Docker Desktop

Viva: Q: Image vs container? — an image is the packaged template; a container is a running instance of it Q: What is a Dockerfile? — a recipe that defines how to build an image Q: What is Docker Compose? — a tool to define and run multi-container applications from a YAML file Q: What does -p 3000:3000 do? — maps host port 3000 to container port 3000

Experiment 7 — Monitoring with Grafana

Aim: monitor system and application metrics using Grafana dashboards.

Do this

28. Download the Grafana installer for your OS and install it.
29. Start the service (Windows: net start Grafana).
30. Open http://localhost:3000 and log in as admin / admin; change the password.
31. Add a data source (for example Prometheus at http://localhost:9090).
32. + Create → Dashboard → Add new panel.
33. Write a query, choose a visualisation (time series, gauge, table), set units and thresholds.
34. Save the dashboard and share the link.
35. Alerting → Alert rules → create a rule (e.g. CPU > 80%) and a contact point.

What to verify / screenshot

- Grafana login
- data source connected
- dashboard with at least 2 panels
- an alert rule

Viva: Q: Grafana vs Prometheus? — Prometheus collects/stores metrics; Grafana visualises and alerts on them Q: What is a data source? — the backend Grafana queries, e.g. Prometheus, InfluxDB, MySQL Q: What is a panel? — a single visualisation inside a dashboard Q: What language is used for Prometheus queries? — PromQL

Experiment 8 — AWS Lambda serverless function

Aim: create and deploy a serverless function using AWS Lambda and execute it on a schedule.

Do this

36. AWS Console → Lambda → Create a function.
37. Author from scratch, name it (e.g. myStopinato), runtime Python 3.x, create.
38. Add trigger → EventBridge (CloudWatch Events) → create a new rule → name everyMinute → schedule expression rate(1 minute) → Add.
39. Code tab → paste the boto3 function → Deploy.
40. Test → configure a test event → Test. Check the result and CloudWatch logs.
41. Monitor tab → show invocations, duration and errors.

Function code

```
import boto3
region = 'us-east-1'
instances = ['<REPLACE_WITH_INSTANCE_ID>']
ec2 = boto3.client('ec2', region_name=region)

def lambda_handler(event, context):
    ec2.stop_instances(InstanceIds=instances)
    print('stopped your instances: ' + str(instances))
```

What to verify / screenshot

- function created
- EventBridge trigger attached
- successful test
- Monitor metrics
- CloudWatch log output

Viva: Q: What is serverless? — you run code without managing servers; the cloud provider scales and bills per execution Q: What is a handler? — the entry function Lambda calls: lambda_handler(event, context) Q: What triggers a Lambda? — events such as API Gateway, S3, EventBridge schedules, queues Q: What is boto3? — the AWS SDK for Python used to call AWS services

Last-minute cheat sheet

Commands one more time

GIT : git init | git add . | git commit -m "msg" | git branch -M main | git remote add origin URL | git push -u origin main

JENKINS : java -jar jenkins.war --httpPort=8080 -> localhost:8080

K8S : kubectl create deployment nginx --image=nginx | kubectl expose ... --type=NodePort --port=80 | kubectl scale ... --replicas=3

DOCKER : docker compose watch | docker build -t welcome . | docker run -d -p 3000:3000 welcome | docker ps

If something breaks

- Port busy → use another port (8080 → 8081, 3000 → 3001).
- Jenkins locked out → read ~/.jenkins/secrets/initialAdminPassword.
- Git push rejected → git pull --rebase origin main, then push.
- Webhook not firing → Jenkins must be publicly reachable; use ngrok.
- Pods not Running → kubectl describe pod <name> and kubectl logs <name>.
- Docker daemon error → start Docker Desktop first.

Order to revise tonight (about 90 minutes)

- 20 min: Exp 1 + 2 — click through Jira, run the Git commands once.
- 20 min: Exp 3 + 4 — understand the Jenkins job types and the pipeline stages.
- 20 min: Exp 5 — run the kubectl commands on Killercoda.
- 15 min: Exp 6 + 7 — start a container, open Grafana.
- 15 min: Exp 8 — read the Lambda steps and the handler code.

Rule for the exam: say the aim, do the steps in order, show the verification (console output / dashboard / pod list), then state the conclusion in one line.